



微分几何引论

曲线与曲面

作者：张三

时间：December 29, 2025



ElegantLaTeX Program

目录

第 1 章 过滤	1
1.1 定理示例	1
1.2 复制文本	1
第 2 章 选择	2
2.1 f1	2
2.2 f2	6
2.3 python code	7
第 3 章 chatu	9
第 4 章 表格	10
第 5 章 数学公式	11

第 1 章 过滤

1.1 定理示例

这是一个重要定理。Update (2024-09-08): Read this article in Japanese at Qiita or at Emacs-JP. 更新 (2024-09-08): 在 Qiita 或 Emacs-JP 上阅读这篇文章

You're using Avy wrong. 你用 Avy 的方式不对。

Too harsh? Let me rephrase: you're barely using Avy. Still too broad? Okay, the noninflammatory version: Avy, the Emacs package to jump around the screen, lends itself to efficient composable usage that's obscured by default. 太严厉了? 让我换个说法: 你几乎没用 Avy。还是太宽泛了? 好吧, 非炎症版本: Avy, Emacs 的屏幕跳跃包, 适合高效的组合使用, 但默认情况下被隐藏了。

Without burying the lede any further, here's a demo that uses a single Avy command (avy-goto-char-timer) to do various things in multiple buffers and windows, all without manually moving the cursor: 不再埋没主要线索, 这里有一个演示, 使用单个 Avy 命令 (avy-goto-char-timer) 在多个缓冲区和窗口中做各种事情, 而无需手动移动光标:

1.2 复制文本

复制文本, 删除行或区域, 移动文本, 标记文本, 调出帮助缓冲区, 查找定义, 搜索谷歌, 检查我的拼写, 等等。我再次强调: Avy 定义了数十个跳跃命令, 但我只用了其中一个。这篇文章详细解释了如何创建你自己的版本, 但更重要的是试图解释为什么这是一个好主意。

This is the first of two parts in a series on Avy, an Emacs package for jumping around efficiently with the keyboard. Part 1 is about about supercharging built-in customization to do anything with Avy, or some approximation thereof. Part 2 will be a more technical (elisp-y) dive into writing more complex features for your individual needs. If you are interested in the short elisp snippets in this document, they are collated into a single file here. 这是关于 Avy 这个 Emacs 插件的两部分系列文章中的第一部分, Avy 是一个用于通过键盘高效跳转的 Emacs 插件。第一部分是关于超级增强内置自定义功能, 用 Avy (或者其某种近似) 来做任何事情。第二部分将更深入地探讨为你的个人需求编写更复杂功能的 (elisp) 技术细节。如果你对这个文档中的简短 elisp 代码片段感兴趣, 它们已经被整理到一个单独的文件中。

第 2 章 选择

2.1 fl

We see the same pattern repeated in most interactions with Emacs whose primary purpose isn't typing text. To perform a task, we Filter, Select and Act in sequence: 我们在大多数不是以文本输入为主要目的的 Emacs 交互中看到同样的模式重复出现。为了执行一个任务，我们按顺序进行过滤、选择和执行：

Filter: Winnow a large pile of candidates to a smaller number, usually by typing in some text. These candidates can be anything, see below for examples. 过滤：将大量候选者筛选成较小的数量，通常通过输入一些文本来实现。这些候选者可以是任何东西，下面是一些例子。

Select: Specify a candidate as the one you need, usually with visual confirmation. If you filter a list down to one candidate, it's automatically selected. 选择：指定您需要的候选者，通常需要视觉确认。如果您将列表筛选至一个候选者，它将自动被选中。

Act: Run the task with this candidate as an argument. 执行：使用此候选者作为参数运行任务。

Want to open a file? Invoke a command, then type in text to filter a list of completions, select a completion, find-file. 想要打开文件？调用命令，然后输入文本以筛选补全列表，选择一个补全项，找到文件。Switch buffers? Type to filter, select a buffer, switch. 切换缓冲区？输入文本以筛选，选择一个缓冲区，切换。Autocomplete a symbol in code? Type a few characters to narrow a pop-up list, choose a candidate, confirm. 代码中自动补全符号？输入几个字符来缩小弹出列表，选择一个候选项，确认。

As ever, this model is a simplification. Helm, Ivy, Dired & co let you select multiple candidates to act on, for instance. We will put this qualification aside while we explore this idea. 如同以往，这个模型是一种简化。Helm、Ivy、Dired 等允许你选择多个候选项来执行操作，例如。我们将暂时放下这个限定，以便我们探索这个想法。

This observation leads to several interesting ideas. For one, the Filter → Select → Act process is often thought of as a single operation. Decoupling the filtering, selection and action phases (in code and in our heads) frees us to consider a flock of possibilities. Here's minibuffer interaction: 这一观察引出了一些有趣的想法。首先，过滤 → 选择 → 执行的过程通常被认为是一个单一操作。将过滤、选择和执行阶段（在代码和在我们的头脑中）解耦，使我们能够考虑一大堆可能性。这是 minibuffer 交互：

When you type (after C-s), you automatically filter buffer text and select the nearest match. You can select each subsequent match in turn, or jump to the first or last match. The Act here is the process of moving the cursor to the match location, but it can be one of a few things, like running occur or query-replace on the search string. Many Isearch commands simultaneously filter, select and act, so we're fitting a square peg in a round hole here². 当你输入（在 C-s 之后），你会自动过滤缓冲区文本并选择最近的匹配项。你可以依次选择每个后续匹配项，或跳转到第一个或最后一个匹配项。这里的 Act 是将光标移动到匹配位置的过程，但它可以是几件事情中的一个，比如在搜索字符串上运行 occur 或 query-replace。许多 Isearch 命令同时过滤、选择和执行操作，所以我们在这里是削足适履²。

If you've spent any time using Isearch, you can appreciate the tradeoff involved in dividing a task into these three independently configurable phases. Lumping two or all three into a single operation makes Isearch a wicked fast keyboard interaction. When I use Isearch my brain is usually trying to catch up to my fingers. On the other hand, the advantage of modularity is expressive power. The three phase process is slower on the whole, but we can do a whole lot more by plugging in different pieces into each of the Filter, Select and Act slots. To see this, you have only to consider how many disparate tasks the minibuffer handles, and in how many different ways! 如果你使用过 Isearch，就能体会到将任务分为这三个独立配置阶段所涉及的权衡。将两阶段或全部三个阶段合并为一个操作，可以让 Isearch 成为一项非常快速的键盘交互。当我使用 Isearch 时，我的大脑通常都在努力跟上我的手指。另一方面，模块化的优势在于表达能力。虽然三个阶段整个过程较慢，但我们可以通过将不同的部件插入到过滤、选择和执行槽中来完成更多的事情。要看到这一点，你只需要考虑小缓冲区处理了多少不同的任务，以及有多少不同的方式！

But back to Isearch: what can we do to decouple the three stages here? Not much without modifying its guts. It's all elisp, so that's not a tall order. For example, Protesilaos Stavrou adds many intuitive actions (such as marking candidates) to Isearch in this video. But it turns out we don't need to modify Isearch, because Avy exists, has a killer Filter feature, and it separates the three stages like a champ. This makes for some very intriguing possibilities. 但回到 Isearch: 我们能做些什么来解耦这三个阶段呢? 如果不修改其内部结构, 那就做不了太多。它全部是用 elisp 编写的, 所以这并不是什么艰巨的任务。例如, Protesilaos Stavrou 在这个视频中为 Isearch 添加了许多直观的操作 (例如标记候选)。但事实证明, 我们不需要修改 Isearch, 因为 Avy 存在, 具有杀手级的过滤功能, 并且像冠军一样分离了这三个阶段。这使得出现了一些非常有趣的可能性。Wait, what's Avy? 等等, 什么是 Avy?

Avy is authored by the prolific abo-abo (Oleh Krehel), who also wrote Ivy, Counsel, Ace-Window, Hydra, Swiper and many other mainstays of the Emacs package ecosystem that you've probably used. If you're reading this, chances are you already know (and probably use) Avy. So here's a very short version from the documentation: Avy 由多产的 abo-abo (Oleh Krehel) 编写, 他还编写了 Ivy、Counsel、Ace-Window、Hydra、Swiper 和许多其他您可能已经使用过的 Emacs 软件包生态系统中的主要软件。如果您正在阅读此内容, 那么您很可能已经知道 (并且可能正在使用) Avy。所以这里是从文档中的一个非常简短的版本:

avy is a GNU Emacs package for jumping to visible text using a char-based decision tree. avy 是一个 GNU Emacs 软件包, 用于使用基于字符的决策树跳转到可见文本。

You can call an Avy command and type in some text. Any match for this text on the frame (or all Emacs frames if desired) becomes a selection candidate, with some hint characters overlaid on top. Here I type in "an" and all strings in the frame that match it are highlighted: 您可以调用一个 Avy 命令并输入一些文本。在框架 (或如果需要, 在所有 Emacs 框架) 上与此文本匹配的任何字符串都成为选择候选, 并在上面覆盖一些提示字符。我在这里输入 "an", 框架中所有匹配该字符串的内容都被突出显示:

Typing in one of the hints then jumps the cursor to that location. Here I jump to this sentence from another window: 输入一个提示后, 光标会跳转到该位置。这里我从另一个窗口跳到这个句子: Play by play 比赛解说

Typical Avy usage looks something like this: Jump to a location that matches your text (across all windows), then jump back with pop-global-mark (C-x C-SPC). In a later section I go into more detail on jumping back and forth with Avy. Here is a demo of this process where I jump twice with Avy and then jump back in sequence: 典型的 Avy 使用方式看起来是这样的: 跳转到与你的文本匹配的位置 (跨所有窗口), 然后使用 pop-global-mark (C-x C-SPC) 跳回。在后面的部分我将更详细地介绍如何使用 Avy 来回跳转。这里有一个演示这个过程, 我用 Avy 跳转了两次, 然后按顺序跳回: Play by play 逐步说明

At least that's the official description. You can peruse the README for more information, but what I find mystifying is that...至少这是官方描述。你可以查阅 README 以获取更多信息, 但我发现令人费解的是.....Avy's documentation leaves out the best part ...Avy 的文档遗漏了最好的部分

Avy handles filtering automatically and the selection is made through a char-based decision tree. Here's how it fits into our three part interaction model. Avy 自动处理过滤, 选择是通过基于字符的决策树进行的。这是它如何融入我们三部分交互模型中的。Filter, 过滤,

Before you call Avy every text character on your screen is a potential candidate for selection. The possibilities are all laid out for you, and there are too many of them! 在你调用 Avy 之前, 屏幕上的每个文本字符都是选择的一个潜在候选。所有可能性都为你展开, 而且太多了!

You filter the candidate pool with Avy similar to how you would in the minibuffer, by typing text. This reduces the size of the pool to those that match your input. Avy provides dozens of filtering styles. It can: only consider candidates above/below point, only beginnings of words, only the current window (or frame), only whitespace, only beginnings of lines, only headings in Org files, the list goes on. 你可以通过在最小化缓冲区中输入文本来使用 Avy 过滤候选池, 就像在 minibuffer 中一样。这会将池的大小减少到与你的输入匹配的那些。Avy 提供了数十种过滤样式。它可以: 仅考虑点以上的候选/点以下的候选, 仅考虑单词的开头, 仅考虑当前窗口 (或框架), 仅考虑空白, 仅考虑行的开头, 仅考虑 Org 文件中的标题, 等等。Filtering commands in Avy Avy 中的过滤命令

Filtering in Avy is independent of the selection method (as it should be), but there is a dizzying collection of filtering methods. I assume the idea is that the user will pick a couple of commands that they need most often and commit only those to memory. Avy 中的过滤与选择方法独立（这应该是这样），但过滤方法种类繁多，令人眼花缭乱。我猜测的思路是用户将选择他们最常用的几条命令，并只将这些命令牢记在心。

Here's the problem: We want to use our mental bandwidth for the problem we're trying to solve with the text editor, not the editor itself. Conscious decision-making is expensive and distracting. As of now we need to decide on the fly between Isearch and Avy to find and act on things. If you use a fancy search tool like Swiper, Helm-swoop or Consult-line, you now have three options. Having a bunch of Avy commands on top is a recipe for mental gridlock. To that end, we just pick the most adaptable, flexible and general-purpose Avy command (avy-goto-char-timer) for everything. 这里的问题是：我们希望将我们的心理带宽用于我们试图用文本编辑器解决的问题，而不是编辑器本身。有意识的决策是昂贵且分散注意力的。目前，我们需要在飞行中决定在 Isearch 和 Avy 之间选择以查找和操作项目。如果你使用像 Swiper、Helm-swoop 或 Consult-line 这样的高级搜索工具，现在有三个选项。在顶部添加一堆 Avy 命令是导致心理混乱的公式。为此，我们只为所有事情选择最适应、灵活和通用的 Avy 命令（avy-goto-char-timer）。

```
(global-set-key (kbd "M-j") 'avy-goto-char-timer)
```

Further below I make the case that you don't need to make even this decision, you can always use Isearch and switch to Avy as needed. 在下面我会论证你甚至不需要做这个决定，你可以随时使用 Isearch 并根据需要切换到 Avy。

To be clear, this decision cost has to be balanced against the cost of frequent busywork and chronic context switching that Avy helps avoid. There is a case to be made for adapting Avy's flexible filtering options to our needs, and the number of packages that offer Avy-based candidate filtering (for everything from linting errors to buffer selection) attests to this. We will examine this in depth in Part II. 换句话说，这个决定成本需要与 Avy 帮助避免的频繁琐事和慢性上下文切换成本相平衡。有理由认为需要根据我们的需求调整 Avy 的灵活过滤选项，而提供基于 Avy 的候选过滤的软件包数量（从语法检查错误到缓冲区选择）证明了这一点。我们将在第二部分深入探讨这一点。

But in this piece we are interested in a different, much less explored aspect of Avy. 但在这篇文章中，我们感兴趣的是 Avy 的一个不同、且很少被探索的方面。Select, 选择，

Every selection method that Avy offers involves typing characters that map to screen locations. This is quite different from Isearch, where you call isearch-repeat-forward (C-s, possibly with a numerical prefix argument) or the minibuffer, where you navigate a completions buffer or list with C-n and C-p. Avy's selection method is generally faster because it minimizes, by design, the length of the character sequences it uses, and we have ten fingers that can access 40 keys in $O(1)$ time. The fact that we're often looking directly where we mean to jump means we don't need to parse an entire screen of gibberish. Unfortunately for this article, this means using Avy is much more intuitive than looking at screenshots or watching demos. Avy 提供的每一种选择方法都涉及输入映射到屏幕位置的字符。这与 Isearch 不同，Isearch 中你调用 isearch-repeat-forward (C-s, 可能带有数字前缀参数) 或使用 minibuffer, 在 completions 缓冲区或列表中使用 C-n 和 C-p 进行导航。Avy 的选择方法通常更快，因为其设计上最小化了使用的字符序列的长度，而且我们有十个手指可以在 $O(1)$ 时间内访问约 40 个键。我们通常直接跳转到我们想要的位置，这意味着我们不需要解析整个屏幕的乱码。不幸的是，对于这篇文章来说，使用 Avy 比看截图或观看演示更直观。

This excellent design leaves us with little reason to tinker with the selection phase: it's sufficiently modular and accommodating of different filter and act stages. You can customize avy-style if you want to change the set or positions of characters used for selection. Here is an example of using simple words to select candidates: 这个出色设计让我们几乎没有理由去调整选择阶段：它足够模块化，能够适应不同的过滤和执行阶段。如果你想改变用于选择的角色集或位置，可以自定义 avy-style。这里是一个使用简单词汇选择候选人的例子：

We will pay more attention to the selection operation in part II as well. 我们在第二部分也会更多地关注选择操作。Act! 行动！

This brings us to the focus of this article. The stated purpose of Avy, jumping to things, makes it sound like a (contextually) faster Isearch. But jumping is only one of many possibilities. Avy provides a "dispatch list", a collection of actions you can perform on a candidate, and they are all treated on equal footing with the jump action. You can show

these commands any time you use Avy with `?`: 这就引出了本文的重点。Avy 的声明目的是跳转到项目，听起来像是一个（上下文中）更快的 Isearch。但跳转只是许多可能性中的一种。Avy 提供了一个“分发列表”，一组可以对候选项目执行的操作，它们都与跳转操作处于平等的地位。你可以在使用 Avy 时随时通过 `?` 显示这些命令：

This means we are free to leverage Avy's unique filtering and selection method to whatever action we want to carry out at any location on the screen. Our interaction model now ends in a block that looks something like this: 这意味着我们可以利用 Avy 独特的过滤和选择方法来执行屏幕上任何位置的任何操作。我们的交互模型现在以一个看起来像这样的代码块结束：

Additionally, Avy also defines a few commands that run different actions, like copying text from anywhere on screen: 此外，Avy 还定义了一些执行不同操作的命令，例如从屏幕上的任何位置复制文本：

Kill 删除 Copy 复制 Move 移动 avy-kill-ring-save-whole-line 复制整行 avy-copy-line 复制当前行 avy-move-line 移动当前行 avy-kill-ring-save-region avy 剪切区域保存 avy-copy-region avy 复制区域 avy-move-region avy 移动区域 avy-kill-region avy 剪切区域 avy-transpose-lines-in-region 交换区域内的行 avy-kill-whole-line 删除整行 avy-org-refile-as-child 将 org 模块作为子模块重新组织

The problem with this approach is that it doesn't scale. Each of these commands defines a full Filter $\rightarrow$ Select $\rightarrow$ Act process, and we quickly run out of headspace and keyboard room if we want any kind of versatility or coverage. They're also not dynamic enough: you're locked into the pipeline and cannot change your mind once you start the command. 这种方法的问题是它无法扩展。每个命令都定义了一个完整的 Filter $\rightarrow$ Select $\rightarrow$ Act 过程，如果我们想要任何种类的多功能性或覆盖范围，我们会很快用完空间和键盘空间。它们也不够动态：一旦开始命令，你就会被锁定在管道中，无法改变主意。

Folks love Vim's editing model for a reason: it's a mini-language where knowing M actions (verbs) and N cursor motions gives you $M \times N$ composite operations. This $(M + N) \rightarrow (M \times N)$ ratio pays off quadratically with the effort you put into learning verbs and motions in Vim. Easymotion, which is Vim's version of Avy3, has part of this composability built-in as a result. We seek to bring some of this to Avy, and (because this is Emacs) do a lot more than combining motions with verbs. We won't need to step into Avy's source code for this, it has all the hooks we need already. 大家喜欢 Vim 的编辑模式是有原因的：它是一个微型语言，知道 M 个动作（动词）和 N 个光标移动就能让你得到 $M \times N$ 个复合操作。这种 $(M + N) \rightarrow (M \times N)$ 的比例随着你对 Vim 中动词和移动的学习投入呈平方级增长。Easymotion 是 Vim 版本的 Avy 3，因此它内置了部分这种可组合性。我们希望将其中一些特性带到 Avy 中，并且（因为这是 Emacs）做比将移动与动词组合更多的事情。我们不需要进入 Avy 的源代码来实现这一点，它已经提供了我们需要的所有钩子。Avy actions Avy 操作

The basic usage for running an arbitrary action with Avy is as follows: 使用 Avy 运行任意操作的基本用法如下：

Call an Avy command. Any command will do, I stick to avy-goto-char-timer. 调用 Avy 命令。任何命令都可以，我坚持使用 avy-goto-char-timer。Filter: Type in some text to shrink the candidate pool from the entire screen to a few locations. 过滤：输入一些文本，将整个屏幕的候选池缩小到几个位置。Act: Specify the action you want to run. You can pull up the dispatch help with `?`, although you won't have to if you set it up right, see Remembering to Avy. 执行：指定你想要执行的操作。你可以使用 `?` 调出分发帮助，虽然如果你设置得当，就不需要，参见记住使用 Avy。Select: Pick one of the candidates to run the action on. 选择：从候选池中选择一个要执行操作的对象。

Here are some things I frequently do with Avy. Note that you can do this on any text in your frame, not just the active window! 这里是我经常用 Avy 做的一些事情。请注意，你可以在任何文本中进行这些操作，而不仅仅是活动窗口！

First, taking the annoyance out of some common editing actions with Avy. If you use Vim and Easymotion, you get the first few actions below for free: 首先，用 Avy 消除一些常见编辑操作的不便。如果你使用 Vim 和 Easymotion，你将免费获得以下几个操作：A note about these demos 关于这些演示的说明

For clarity, I set Avy to desaturate the screen and to “pulse” the line during a few of these actions. These are not enabled by default. I also slowed down the operations by adding a delay to make them easier to follow. In actual usage these are instantaneous. 为了清晰，我将 Avy 设置为降低屏幕饱和度并在执行其中一些操作时“脉冲”该行。这些

默认情况下是禁用的。我还通过添加延迟来减慢了操作速度，以便更容易跟随。在实际使用中，这些操作是即时完成的。

The keys Avy uses to dispatch actions on candidates are specified in `avy-dispatch-alist`. Avy 用于在候选对象上分发操作的键指定在 `avy-dispatch-alist` 中。

We will also need to ensure that these keys don't coincide with the ones Avy uses as selection hints on screen. Consider customizing `avy-keys` for this. 我们还需要确保这些键不与 Avy 在屏幕上用作选择提示的键相冲突。考虑自定义 `avy-keys`。

Kill a candidate word, sexp or line 删除一个候选词、sexp 或行

Killing words or s-expressions is built-in. I added an action to kill a line. In this demo I quickly squash some typos and delete a comment, then remove some code in a different window: 删除单词或 s-expressions 是内置的。我添加了一个操作来删除一行。在这个演示中，我快速修正了一些拼写错误并删除了一条注释，然后在不同的窗口中删除了一些代码：Play by play 逐步说明

```
(defun avy-action-kill-whole-line (pt) (save-excursion (goto-char pt) (kill-whole-line)) (select-window (cdr (ring-ref avy-ring 0))))
```

1.

```
(setf (alist-get ?k avy-dispatch-alist) 'avy-action-kill-whole-line (alist-get ?K avy-dispatch-alist) 'avy-action-kill-whole-line)
```

Yank a candidate word, sexp or line 拉取候选词、sexp 或行

2.2 f2

Copy to the kill-ring or copy to point in your buffer. In this demo I copy some text from a man page into my file: 复制到剪贴板或复制到缓冲区的点。在这个演示中，我从手册中复制了一些文本到我的文件：Play by play 逐步说明

```
(defun avy-action-copy-whole-line (pt)
  (save-excursion
    (goto-char pt)
    (cl-destructuring-bind (start . end)
      (bounds-of-thing-at-point 'line)
      (copy-region-as-kill start end)))
  (select-window
    (cdr
      (ring-ref avy-ring 0)))
  t)

(defun avy-action-yank-whole-line (pt)
  (avy-action-copy-whole-line pt)
  (save-excursion (yank))
  t)

(setf (alist-get ?y avy-dispatch-alist) 'avy-action-yank
      (alist-get ?w avy-dispatch-alist) 'avy-action-copy
      (alist-get ?W avy-dispatch-alist) 'avy-action-copy-whole-line
      (alist-get ?Y avy-dispatch-alist) 'avy-action-yank-whole-line)
```

Note that Avy actually defines separate commands for this: `avy-copy-line` and `avy-copy-region` to copy lines and regions from anywhere in the frame. These are a little faster since they have the action stage baked into the function

call. You might be better served by these. But we want to avoid the mental burden of remembering too many top level commands, so we work in two simpler stages: call `avy-goto-char-timer` (to filter and select) and then dispatch on our selected candidate as we see fit. 注意 Avy 实际上为这个定义了不同的命令: `avy-copy-line` 和 `avy-copy-region` 来从缓冲区的任何地方复制行和区域。这些稍微快一点, 因为它们将操作阶段嵌入到函数调用中。你可能更适合使用这些。但我们想避免记住太多顶级命令的精神负担, 所以我们分为两个更简单的阶段: 调用 `avy-goto-char-timer` (过滤和选择), 然后根据我们选择的内容适当地分发。Move a candidate word, sexp or line 移动候选词、sexp 或行

Avy calls this “teleport”, I call it “transpose”, either way it’s bound to `t`. In this demo I move text around the buffer …without moving (much) around the buffer: Avy 称之为“传送”, 我称之为“转置”, 无论如何它都绑定到 `t`。在这个演示中我移动缓冲区中的文本……而几乎没有在缓冲区中移动: Play by play 逐步说明

```
(defun avy-action-teleport-whole-line (pt) (avy-action-kill-whole-line pt) (save-excursion (yank)) t)
(setf (alist-get ?t avy-dispatch-alist) 'avy-action-teleport (alist-get ?T avy-dispatch-alist) 'avy-action-teleport-whole-line)
```

Zap to a candidate position 快速定位到候选位置

This is built-in and bound to `z` by default: 这是默认绑定到 `z` 的内置功能: Play by play 逐步说明

Mark a candidate word or sexp 标记候选单词或 sexp

Also built in, `m` by default. This isn’t different from jumping to the candidate using Avy and calling `mark-sexp`, but it is more convenient: 默认情况下内置了 `m`。这与使用 Avy 跳转到候选并调用 `mark-sexp` 没有区别, 但更方便: Your browser does not support the video tag. Play by play 逐步说明

Mark the region from point to a candidate 从一点到候选点标记区域

Avy sets the mark before it jumps, so you could use `C-x C-x` to activate the region, but this saves you the trouble. Avy 在跳跃前设置标记, 所以你可以使用 `C-x C-x` 来激活区域, 但这会为你节省麻烦。Your browser does not support the video tag. Play by play 逐步说明

```
(defun avy-action-mark-to-char (pt) (activate-mark) (goto-char pt))
(setf (alist-get ? avy-dispatch-alist) 'avy-action-mark-to-char)
```

Next, some contextual actions automagicked by Avy: 接下来, 一些由 Avy 自动完成的上下文操作: `ispell a candidate word` 检查候选词

This is built-in, bound to `i` by default. 这是内置的, 默认绑定到 `i`。

2.3 python code

```
#
#
# This program is free software; you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.

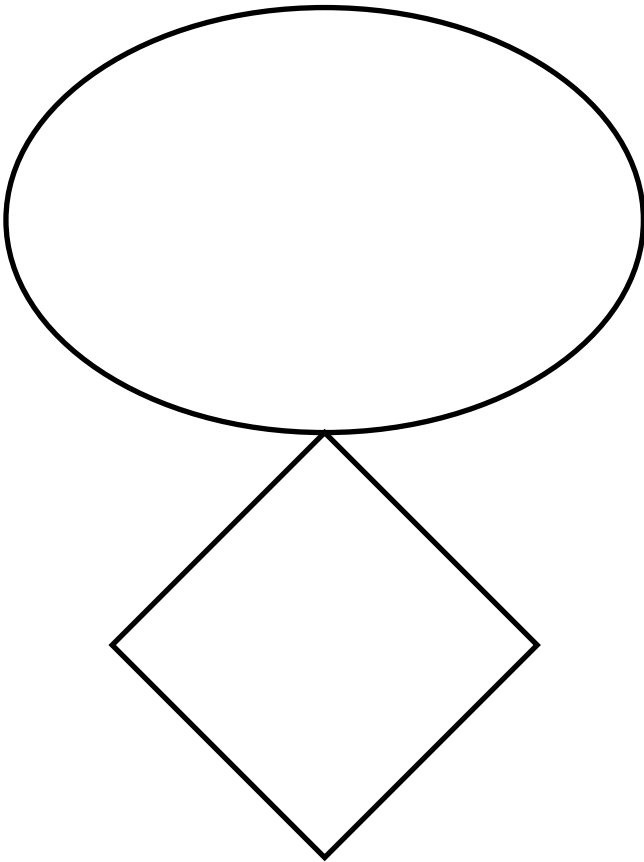
def find():
    j = 10
    for i in range(0, 10):
        print(j + i)

def methodname(self, arg):
    pass
```

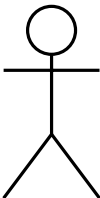
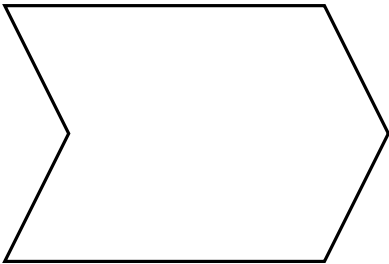
```
if __name__ == "__main__":  
    f = 100  
    print("free_start")  
    find()  
    print(f)
```

第 3 章 chatu

sdjfsajf



sjdfkasjf



Actor

第 4 章 表格

表 4.1: 表 1

name	score
donald	100
iris	99

第 5 章 数学公式

$$f(x) = 10x$$

我来看看 斜体字 粗体字。